

MDI3003

ADVANCED PREDICTIVE ANALYTICS

LABORATORY MANUAL – LAB 04

Probabilistic Customer Segmentation and Segment Prediction Using Demographic, Psychographic, and Behavioral Data with Naive Bayes Classifiers

Dr. Durgesh Kumar

Assistant Professor (Senior), SCOPE, VIT Vellore

Scheduled Duration: 3 Hours (Core) | Research Extension: 3–5 Hours / instructor-defined window

Technical positioning: Naive Bayes is a supervised classification method. Therefore, this laboratory predicts predefined customer-segment labels. It does not discover segments through unsupervised clustering.

Student Name: _____

Registration Number: _____

Date: _____ Batch: _____

Revision History

Revision 3.0 (August 2026): Added verified multi-dataset options and advanced Transformer-based customer modeling (TabTransformer/FT-Transformer; optional BERT for genuine text fields).

How to Use This Manual

- Read Sections 1–13 before the laboratory session.
- Complete the 3-hour core workflow in the stated order and obtain all four instructor check-offs.
- Use the supplied code as a scaffold; replace configurable paths and column names with the instructor-provided dataset schema.
- Do not inspect or tune against the locked test set until model selection is complete.
- Submit the notebook, report, result files, figures, model artifacts, and README using the specified filenames.
- Complete optional extension experiments only after the core workflow is reproducible.
- Use only the instructor-issued dataset pack for the assessed core lab; record its checksum, schema version, label-definition note, and data dictionary before modeling.
- Core models are DummyClassifier, GaussianNB, BernoulliNB, and a fully specified mixed-feature CategoricalNB. ComplementNB, Logistic Regression, repeated-CV uncertainty, fairness, calibration, and temporal-shift studies are extensions.
- Do not claim that a small metric difference proves superiority: report fold-to-fold variability and, in the extension, a confidence interval for the final comparison.

Academic integrity: External code or AI assistance must be acknowledged. Students remain responsible for understanding, testing, and explaining every submitted component.

Contents

1. Laboratory positioning
2. Aim and objectives
3. Learning outcomes
4. Problem statement
5. Business context
6. Scope and assumptions
7. Dataset and data governance
8. Feature taxonomy
9. Theoretical background
10. Naive Bayes variants
11. End-to-end workflow
12. Software and hardware requirements
13. Three-hour plan and check-offs
14. Step-by-step laboratory procedure
15. Model evaluation
16. Feature-group experiments
17. Probability and confidence analysis
18. Business-critical error analysis
19. Visualization requirements
20. Result-table templates

- 21. New-customer prediction
- 22. Discussion questions
- 23. Viva questions
- 24. Common mistakes
- 25. Responsible analytics
- 26. Submission guidelines
- 27. Assessment rubric
- 28. Final checklist
- 29. References and student-help links
- 30. Appendices

1. Laboratory Positioning

This laboratory develops a leakage-safe, reproducible, supervised customer-segment prediction system. The three-hour core deliberately prioritizes a small set of technically compatible baselines (DummyClassifier, GaussianNB, BernoulliNB, and mixed-feature CategoricalNB) on one fixed instructor-issued dataset. ComplementNB and broader robustness studies are research extensions. The final model is selected using training-only validation evidence and evaluated once on a locked test set.

International benchmark alignment: The design uses baseline-first experimentation, staged check-offs, comparative evaluation, class-wise error analysis, reproducibility artifacts, and responsible-computing reflection. These are recurring practices in rigorous machine-learning and data-science coursework at leading universities.

2. Aim and Objectives

2.1 Aim

To build, compare, and interpret supervised Naive Bayes classifiers that predict predefined customer segments from demographic, psychographic, and behavioral attributes.

2.2 Objectives

- Audit a mixed-type customer dataset for quality, imbalance, privacy, and potential leakage.
- Construct feature groups for demographic, psychographic, and behavioral variables.
- Implement a DummyClassifier baseline plus GaussianNB, BernoulliNB, and a complete mixed-feature CategoricalNB in the core; evaluate ComplementNB only as an explicitly justified sparse non-negative extension.
- Use leakage-safe pipelines and a consistent split for all comparative models.
- Select the final model using mean cross-validation macro F1 and secondary operational criteria.
- Evaluate the selected model on unseen data using class-wise and aggregate metrics.
- Analyze posterior probabilities, low-confidence predictions, and business-critical errors.
- Translate findings into responsible, evidence-based business recommendations.

3. Learning Outcomes

- Distinguish supervised segment classification from unsupervised customer clustering.
- Explain priors, likelihoods, posteriors, smoothing, and the conditional-independence assumption.
- Choose a Naive Bayes variant compatible with the transformed feature representation.
- Create leakage-safe preprocessing for numerical, categorical, and binary data.
- Report macro F1, weighted F1, class-wise metrics, confusion matrices, fold-to-fold variability, and—during the extension—a bootstrap 95% confidence interval for final macro F1.
- Inspect model confidence without treating probability as certainty.
- Identify privacy, fairness, stereotyping, and misuse risks in customer profiling.
- Produce a reproducible notebook and concise technical report.

4. Problem Statement

A business maintains customer records containing demographic, psychographic, and behavioral characteristics. Each customer has an existing segment label created through an approved business process. The task is to predict the predefined segment for a new customer while preserving privacy, evaluating minority classes, and documenting model limitations.

Label rule: Use only labels present in the instructor-provided dataset. Example segment names in this manual are illustrative and must not be substituted for the actual target values.

5. Business Context

Responsible segment prediction can support campaign planning, customer engagement, retention analysis, and service personalization. A prediction is not a verified customer preference and must not be used as the sole basis for discriminatory pricing, exclusion, denial of service, or manipulation.

Business use	Potential value	Risk if misused
Campaign targeting	More relevant communication	Stereotyping or excessive profiling
Retention support	Earlier identification of disengagement	False at-risk labels may waste resources
Service personalization	Improved channel or content selection	Sensitive inference and unequal treatment
Portfolio analysis	Better understanding of segment mix	Historical labels may encode bias

6. Scope and Operational Assumptions

- The target variable contains at least two predefined customer segments.
- Direct identifiers are excluded from model features.
- All models within a dataset use the same saved split.
- Preprocessing is fitted only on training data or training folds.
- The test set remains locked until model selection is finished.
- The core laboratory uses DummyClassifier, GaussianNB, BernoulliNB, and CategoricalNB. ComplementNB, Logistic Regression, calibration, quantitative fairness, and temporal/domain-shift analyses are extensions.
- Business recommendations are descriptive and reviewable, not autonomous decisions.

7. Dataset and Data Governance

7.1 Minimum schema

Feature group	Illustrative variables	Typical type
Identifier	customer_id	Identifier; exclude from predictors
Demographic	age, education, occupation, income, family_size, region	Continuous or categorical
Psychographic	lifestyle, price_sensitivity, brand_consciousness, technology_affinity	Categorical, ordinal, or score
Behavioral	purchase_frequency, recency, total_spending, campaign_response, discount_usage	Continuous, count, or binary
Target	customer_segment	Categorical class label

7.2 Dataset card

- Dataset name and source
- Licence or permission
- Record and feature counts
- Class labels and imbalance
- Missingness and duplicates
- Sensitive attributes
- Sampling and collection limitations
- Privacy and intended-use statement
- Dataset hash or version identifier

7.3 Excluded direct identifiers

7.4 Fixed Instructor Dataset Pack

For the assessed core lab, the instructor must issue one fixed dataset pack before the session. The pack should contain: customer_segmentation.csv (or equivalent), data_dictionary.csv, dataset_card.md, label_definition.md, and a SHA-256 checksum or version identifier. Students must not silently substitute a different Kaggle or public dataset for the assessed experiment.

7.5 Label-Provenance and Circularity Audit

Before modelling, students must document how customer_segment was created. Flag any predictor that directly encodes the historical segmentation rule, was measured after the segment assignment, or is a deterministic proxy for the label. Such variables can create circular prediction and misleadingly high validation performance.

7.6 Psychographic Measurement Provenance

For each psychographic variable, record whether it is self-reported, survey-derived, inferred from behaviour, or externally assigned. Discuss measurement reliability and whether the construct can change over time. Inferred psychographic attributes require additional caution because they may amplify profiling and fairness risks.

- Full name
- Email address
- Telephone number
- Postal address
- Government identifier
- Payment-card or account number

7.7 Verified Public Dataset Options

The following datasets were externally verified as existing public resources. Their suitability differs, so students must not treat them as interchangeable. The instructor should freeze one primary dataset/version for the core laboratory and record the access date, licence, schema, and checksum.

Dataset A - JanataHack Customer Segmentation (Analytics Vidhya / verified Kaggle mirror)

Direct suitability: HIGH. The automobile-customer dataset contains demographic and socioeconomic attributes including Gender, Ever_Married, Age, Graduated, Profession, Work_Experience, Spending_Score, Family_Size, Var_1, and a multiclass target Segmentation with classes A, B, C, and D. It is the recommended core dataset when a reproducible classroom copy is supplied. Verified sources:

<https://www.analyticsvidhya.com/datahack/contest/janatahack-customer-segmentation> and
<https://www.kaggle.com/datasets/vetrirah/customer>

Dataset B - Customer Personality Analysis (Kaggle)

Direct suitability: MODERATE. This verified dataset contains rich demographic and behavioral marketing attributes and is useful for profiling, feature engineering, campaign-response prediction, or an instructor-defined segmentation extension. It does not provide the same predefined A-D target as Dataset A. Students must not fabricate segment labels. If used here, the instructor must supply a frozen documented target or explicitly redefine the supervised task. Verified source: <https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis>

Dataset C - UCI Online Retail II

Direct suitability: RESEARCH EXTENSION. UCI verifies Online Retail II as a real two-year UK online-retail transaction dataset containing 1,067,371 transactions, with invoice, product, quantity, price, customer, timestamp, and country information. It is well suited to deriving behavioral features such as recency, frequency, and monetary value, but it does not provide predefined customer-segment labels. Use it only in a two-stage extension with a frozen, documented label-construction procedure. DOI: 10.24432/C5CG6D. Source:

<https://archive.ics.uci.edu/dataset/502/online+retail+ii>

Dataset D - UCI Bank Marketing (optional related benchmark)

Direct suitability: RELATED SUPERVISED BENCHMARK, not a segment dataset. It can be used to practice mixed-type preprocessing and classification because its supervised target records whether a client subscribed to a term deposit. Describe this task as response prediction, not customer segmentation. Verified source: <https://archive.ics.uci.edu/dataset/222/bank+marketing>

Dataset-use rule

For the assessed core, prefer Dataset A because it directly contains predefined segment labels. Datasets B-D are alternatives or extensions only when their target definition matches the stated learning objective. Never merge these datasets into one training set because their schemas, populations, targets, and data-generation processes differ.

8. Feature Taxonomy

Category	Meaning	Examples	Analytical caution
Demographic	Observable population characteristics	Age, education, occupation, household size	May include sensitive or protected attributes
Psychographic	Attitudes, interests, values, and preferences	Lifestyle, price sensitivity, technology affinity	Record whether self-reported, survey-derived, or behaviorally inferred; assess measurement reliability and profiling risk.
Behavioral	Observed interactions and transactions	Recency, frequency, spending, campaign response	Can drift over time and reflect platform design
Target label	Existing business segment	Loyal, premium, at-risk, promotion-sensitive	Historical labels can encode policy and bias

9. Theoretical Background

9.1 Classification versus clustering

Classification learns from labeled examples and predicts one of the known segment labels. Clustering discovers groups without target labels. Because Naive Bayes is supervised, this laboratory is a segment-classification exercise.

9.2 Bayes theorem

For class C_k and customer feature vector x :

$$P(C_k | x) = P(x | C_k) P(C_k) / P(x)$$

The prior $P(C_k)$ represents the class frequency before observing the customer. The likelihood $P(x|C_k)$ models how compatible the features are with that segment. The posterior $P(C_k|x)$ is the updated class probability under the model assumptions.

9.3 Naive Bayes decision rule

$$\hat{y} = \operatorname{argmax}_k P(C_k) \prod_j P(x_j | C_k)$$

The “naive” assumption treats features as conditionally independent given the class. This is rarely exactly true, but the model can still be effective when the representation is suitable.

9.4 Evaluation metrics

Metric	Expression/meaning	Why it matters
Precision	$TP/(TP+FP)$	How reliable a predicted segment is
Recall	$TP/(TP+FN)$	How many actual members of a segment are recovered

Metric	Expression/meaning	Why it matters
F1	$2PR/(P+R)$	Balances precision and recall
Macro F1	Mean of class-wise F1	Gives equal weight to minority segments
Weighted F1	Support-weighted class-wise F1	Reflects overall class distribution
Log loss	Penalizes incorrect confident probabilities	Useful when probability quality matters

10. Naive Bayes Classifier Variants

Model	Compatible representation	Key parameter	Main caution
DummyClassifier	Any encoded input	strategy	Must be outperformed by useful models
GaussianNB	Continuous numeric features	var_smoothing	Gaussian and independence assumptions
CategoricalNB	Non-negative integer category codes	alpha	CORE: all category codes must be non-negative integers; reserve a code for unknown values.
BernoulliNB	Binary indicators	alpha, binarize	Information is reduced to presence/absence
ComplementNB	Sparse non-negative features	alpha, norm	EXTENSION: exploratory sparse non-negative benchmark; justify why its text-oriented design is meaningful for this tabular representation.
Optional Logistic Regression	One-hot + scaled numeric features	C, class_weight	Non-Naive-Bayes benchmark only
Representation safety: CategoricalNB requires non-negative integer category codes; the core notebook therefore uses a safe unknown-category mapping. ComplementNB and MultinomialNB require non-negative inputs and must not receive arbitrary standardized values. GaussianNB should be used on continuous numeric variables rather than arbitrary category codes.			

10.1 Advanced Learner Extension - Transformer-Based Customer Models

The advanced extension must use a Transformer architecture suited to the data modality. For structured demographic, psychographic, and behavioral tables, TabTransformer or FT-Transformer is preferred. For genuine free-text fields such as reviews, survey comments, complaints, or call notes, BERT/DistilBERT may be used as a text encoder and fused with tabular features.

A. TabTransformer / FT-Transformer for structured customer data

TabTransformer applies self-attention to learned categorical-feature embeddings to obtain contextual representations for supervised tabular prediction. FT-Transformer is another Transformer adaptation developed for tabular data. Compare the Transformer with the selected Naive Bayes baseline on the same frozen split and report macro F1, weighted F1, per-class recall, training time, inference latency, parameter count, and robustness to missing/noisy inputs. References: https://keras.io/examples/structured_data/tabtransformer/ and <https://proceedings.neurips.cc/paper/2021/hash/9d86d83f925f2149e9edb0ac3b49229c-Abstract.html>

B. Optional BERT/DistilBERT text-augmented model

Use BERT-family models only when meaningful free-text customer information is present. Evaluate text-only and tabular-only ablations, then optionally fuse the BERT representation with processed tabular features. Do not convert

ordinary numeric/categorical columns into artificial sentences merely to justify BERT. Reference: https://huggingface.co/docs/transformers/tasks/sequence_classification

Advanced-learning question

Does a Transformer provide enough predictive or representational benefit to justify its extra complexity over a well-designed Naive Bayes or linear baseline? Answer with controlled experimental evidence, not an assumption that deeper models are automatically superior.

11. End-to-End Workflow

1. Define the approved business objective.
2. Load and validate the dataset.
3. Remove direct identifiers.
4. Audit missingness, duplicates, class balance, and sensitive fields.
5. Assign features to demographic, psychographic, and behavioral groups.
6. Create and save a stratified or grouped split.
7. Build training-only preprocessing pipelines.
8. Train the Dummy baseline.
9. Train the core models: DummyClassifier, GaussianNB, BernoulliNB, and mixed-feature CategoricalNB.
10. Compare demographic, psychographic, behavioral, and combined feature groups using one predeclared core model and identical cross-validation folds; do not exhaustively run every classifier on every feature group during the three-hour slot.
11. Select the final model without using test results.
12. Evaluate once on the locked test set.
13. Inspect class-wise errors and posterior probabilities.
14. Predict new customer profiles with validation and review flags.
15. Document responsible-use boundaries and save all artifacts.

12. Software and Hardware Requirements

Component	Requirement
Python	3.10 or later
Environment	Google Colab, Jupyter Notebook, or approved equivalent
Core libraries	pandas, numpy, scikit-learn, matplotlib, joblib
Optional libraries	seaborn only if institution permits; imbalanced-learn for extension
Hardware	Standard CPU is sufficient for the core lab
Input	Instructor-provided or approved customer dataset
Output	Notebook, PDF report, CSV results, figures, fitted model, README

13. Three-Hour Plan and Instructor Check-offs

Time	Activity	Required evidence
0–20 min	Problem framing, fixed dataset-pack verification, provenance/privacy/label audit	Dataset card, checksum/version, target definition, circularity check
20–40 min	EDA and feature taxonomy	Class distribution, missingness, duplicates, feature-group manifest
40–60 min	Locked split and model-specific preprocessing	Saved split manifest; no overlap; training-only transformations
60–85 min	Dummy baseline and GaussianNB	Baseline and numeric-model CV results

Time	Activity	Required evidence
85–110 min	BernoulliNB and complete mixed-feature CategoricalNB	Compatible non-negative encodings; core-model CV results
110–135 min	Feature-group ablation with one declared core model	Demographic vs psychographic vs behavioral vs combined comparison
135–155 min	Model selection with identical CV folds	Mean macro F1, SD, class-wise considerations, selection note
155–170 min	One-time locked-test evaluation and five error cases	Test metrics, confusion matrices, five interpreted errors
170–180 min	Confidence/review policy, new-profile prediction, artifact check	Validation-selected review rule, prediction output, acceptance tests
Check-off		Evidence
1. Dataset and label integrity		Dataset-pack version/checksum, target definition, class distribution, privacy note, label-provenance/circularity audit.
2. Split and preprocessing		Saved split, no ID/group overlap, preprocessing fit only on training data, CategoricalNB transformed values verified non-negative.
3. Model selection		Dummy + three core NB models, identical folds, CV macro-F1 mean/SD, feature-group ablation, written selection decision before test access.
4. Final evaluation and responsible use		Locked-test metrics, five business-critical errors, validation-selected review threshold, new-profile prediction, reproducibility and fairness caveats.

14. Step-by-Step Laboratory Procedure

14.1 Configure the environment

Set a random seed, create output folders, and record package versions.

14.2 Load and validate data

Confirm the fixed dataset-pack checksum/version, required columns, target classes, data types, customer-ID uniqueness, label-definition note, and data dictionary.

14.3 Remove identifiers

Drop direct identifiers from predictors while retaining customer_id only for traceable output files.

14.4 Audit data quality

Report missingness, exact and near-duplicate profiles, suspicious constants, implausible values, class imbalance, psychographic measurement provenance, and any feature that may be a deterministic or post-outcome proxy for the target label.

14.5 Define feature groups

Create explicit demographic, psychographic, behavioral, numeric, categorical, and binary lists.

14.6 Create the locked split

Use stratification and grouping when household or repeated-customer structure exists.

14.7 Build model-specific preprocessing

Ensure each Naive Bayes variant receives a compatible transformed matrix.

14.8 Train the baseline

Evaluate a most-frequent DummyClassifier first. A useful model must demonstrate clear improvement over this trivial baseline.

14.9 Cross-validate candidate models

Use identical predeclared folds for DummyClassifier, GaussianNB, BernoulliNB, and CategoricalNB. Select using mean macro F1, fold-to-fold variability, class-wise recall, assumption fit, and operational error cost.

14.10 Compare feature groups

Using one predeclared core model, compare demographic-only, psychographic-only, behavioral-only, and combined inputs. Treat this as feature-group ablation, not as an exhaustive model × feature-grid search.

14.11 Select and lock the model

Document the selection before evaluating the test set.

14.12 Evaluate the locked test set

Report aggregate and class-wise metrics, matrices, probabilities, and inference time.

14.13 Analyze errors

Inspect at least five errors and explain operational consequences.

14.14 Predict new profiles

Validate input, return the segment, posterior distribution, confidence category, and review recommendation.

14.15 Save and verify artifacts

Save model, preprocessing, results, figures, manifest, and README; rerun the notebook from a clean state.

15. Model Evaluation

- Use five-fold StratifiedKFold for the core lab.
- Report mean and standard deviation of macro F1.
- Use the same split and folds for all models.
- Use the locked test set only after model selection.
- Report class-wise precision, recall, F1, support, count matrix, and row-normalized matrix.
- Record training and inference time.
- Use log loss only for models with meaningful predict_proba outputs.

Core uncertainty rule: report the five fold-level macro-F1 values, their mean, and standard deviation; do not interpret a very small mean difference as decisive when fold variability overlaps.

Extension uncertainty rule: compute a stratified bootstrap 95% confidence interval for the selected model's locked-test macro F1 (or use repeated stratified CV when test size is too small for a stable bootstrap).

Extension temporal robustness: when timestamps exist, create a historical-to-future holdout or pre/post-period split and compare it with random stratified validation to expose drift or recency leakage.

16. Feature-Group Experiments

Experiment	Input features	Question
------------	----------------	----------

Experiment	Input features	Question
A	Demographic only	How much can static population characteristics explain?
B	Psychographic only	Do attitudes and preferences distinguish the labels?
C	Behavioral only	Do observed actions provide the strongest predictive signal?
D	Combined	Does integrating all groups improve macro F1 and minority-class recall?

Students must compare predictive performance, stability, interpretability, and ethical risk. A higher score does not automatically justify using sensitive demographic features.

16.1 Multi-Dataset and Advanced-Model Extension

Advanced learners should complete at least one controlled secondary-dataset experiment in addition to the core Dataset A study. The aim is to test transferability of conclusions, not to pool incompatible data.

Required extension comparison: (1) selected Naive Bayes baseline on Dataset A; (2) TabTransformer or FT-Transformer on the identical Dataset A split; (3) one additional verified dataset with its task and target explicitly documented; and, only if genuine text exists, (4) optional BERT/DistilBERT text-only or fused model.

Report dataset size, class balance, feature modality, macro/weighted F1, training cost, latency, interpretability, and failure cases. Do not rank models across different datasets using raw scores as though the tasks were identical.

17. Probability and Confidence Analysis

For models that support `predict_proba`, report the highest posterior value and full class distribution. Select a review/abstention threshold only from validation predictions. Build a coverage-error table across candidate thresholds: coverage is the fraction assigned automatically; selective error is the error rate among assigned cases. Choose and justify a threshold that matches the stated business risk, freeze it before the locked test, and do not treat Naive Bayes posterior values as calibrated certainty.

Example validation-selected policy	Action
High confidence	Return prediction with normal human review
Moderate confidence	Return prediction with an explicit review flag
Low confidence	Abstain or route for manual analysis
Interpretation: Naive Bayes probabilities may be poorly calibrated when assumptions are violated. A high posterior is not proof that the customer truly belongs to a segment.	

18. Business-Critical Error Analysis

Example error	Potential consequence	Required analysis
High-value → budget-conscious	Inappropriate offer strategy or lost revenue	Inspect influential attributes and confidence
Loyal → at-risk	Unnecessary retention intervention	Check label noise and recency features
At-risk → loyal	Missed retention opportunity	Prioritize class recall and cost
Promotion-sensitive → premium	Ineffective communication	Inspect discount and purchase-response variables
Minority segment → majority segment	Unequal service or invisible subgroup	Report class-wise recall and support

19. Visualization Requirements

- Class-distribution chart

- Missing-value chart
- Numerical-feature distributions
- Categorical frequency plots
- Spending versus frequency scatter plot when applicable
- Cross-validation comparison
- Feature-group comparison
- Count and row-normalized confusion matrices
- Per-class precision/recall/F1 plot
- Confidence distribution
- Optional calibration or fairness comparison

Every figure must have a descriptive title, axis labels, readable text, a legend where needed, a caption, and a short interpretation grounded in the observed data.

20. Result-Table Templates

Table 1. Dataset profile

Dataset	Source	Records	Features	Segments	Missing %	Licence	Privacy

Table 2. Feature grouping

Feature	Type	Group	Preprocessing	Business meaning

Table 3. Class distribution

Segment	Train count	Test count	Percentage

Table 4. Model configuration

Model	Representation	Parameters	Assumptions

Table 5. Cross-validation

Model	Feature group	Accuracy mean	Macro F1 mean	Macro F1 SD	Weighted F1	Time

Model	Feature group	Accuracy mean	Macro F1 mean	Macro F1 SD	Weighted F1	Time

Table 6. Locked test

Model	Accuracy	Macro precision	Macro recall	Macro F1	Weighted F1	Latency

Table 7. Per-class performance

Segment	Precision	Recall	F1	Support

Table 8. Feature-group comparison

Feature group	Best model	Macro F1	Weighted F1	Observation

Table 9. Error analysis

Customer ID	Actual	Predicted	Confidence	Reason	Consequence

Table 10. New profiles

Profile ID	Predicted segment	Posterior	Review status	Interpretation

Table 11. Final recommendation

Selected model	Evidence	Strengths	Limitations	Suitable use	Unsuitable use

Table 12. Verified dataset comparison

Dataset | Verified source | Task/target | Direct suitability | Records | Feature modalities | Licence/access note | Core/Extension

Table 13. Transformer comparison

Model | Dataset | Representation | Macro F1 | Weighted F1 | Train time | Inference latency | Parameters | Key limitation

Table 14. Text-augmented model (optional)

Model | Text field | Tabular features | Fusion method | Macro F1 | Text-only ablation | Tabular-only ablation | Observation

21. New-Customer Prediction

The final prediction function must validate the input schema and return the predicted label, full probability distribution, confidence category, and manual-review recommendation. It must reject missing mandatory fields, invalid numeric ranges, and unsupported categories where the pipeline cannot safely handle them.

22. Discussion Questions

1. Why is this task supervised rather than unsupervised?
2. When would clustering be more appropriate?
3. Why is accuracy inadequate for imbalanced segments?
4. What does conditional independence mean in this context?
5. Why may demographic variables alone be insufficient?
6. Which feature group gave the strongest evidence, and why?
7. Why might behavioral data outperform psychographic data?
8. How can customer preferences and behavior drift over time?
9. Why must customer identifiers be excluded?
10. When is GaussianNB inappropriate?
11. Why is additive smoothing needed?
12. What causes low-confidence predictions?
13. Which error is most costly to the business?
14. How can historical labels encode bias?
15. Should sensitive demographic variables be used?
16. How can segmentation become discriminatory?
17. Why should predictions support rather than replace human decisions?
18. How should the model be monitored after deployment?
19. Why is TabTransformer more appropriate than BERT for ordinary structured customer tables?
20. When would BERT add real value to a customer-segmentation system?
21. Why should raw performance scores not be compared directly across datasets with different targets and populations?
22. What evidence would justify the additional computational cost of a Transformer over Naive Bayes?

23. Viva Questions and Expected Key Points

Question	Expected key point
----------	--------------------

Question	Expected key point
What is customer segmentation?	Grouping or labeling customers for analysis and action.
Classification versus clustering?	Classification predicts known labels; clustering discovers unlabeled groups.
What is a prior?	Class probability before observing features.
What is a likelihood?	Probability of the observed features under a class.
What is a posterior?	Updated class probability after observing features.
Why “naive”?	Conditional independence of features given class.
GaussianNB use?	Continuous features modeled with class-specific Gaussian distributions.
CategoricalNB use?	Discrete non-negative category indices.
BernoulliNB use?	Binary presence/absence indicators.
ComplementNB use?	Non-negative sparse features, often with imbalance.
What is smoothing?	Adds pseudo-counts to avoid zero likelihoods.
Why stratify?	Preserve class proportions across splits/folds.
What is leakage?	Information from validation/test influences training.
Why macro F1?	Equal importance to each class.
Why weighted F1?	Reflects class support.
What is a confusion matrix?	Counts actual versus predicted classes.
Why a dummy baseline?	Tests whether a model beats trivial prediction.
Why same split?	Ensures fair model comparison.
What is calibration?	Agreement between predicted probabilities and observed frequencies.
What is abstention?	Refusing to predict when confidence is insufficient.
Why exclude IDs?	They may leak identity and do not generalize.
What is label bias?	Historical labels reflect past policy or discrimination.
What is concept drift?	The relationship between features and labels changes over time.
Why human review?	Predictions are uncertain and may affect people.
Main deployment limitation?	Data quality, label validity, drift, fairness, and model assumptions.

24. Common Mistakes

Mistake	Why it is incorrect	Correct practice
Calling Naive Bayes clustering	Naive Bayes requires labels	Describe the task as supervised segment classification
Using unlabeled data	No target exists for supervised learning	Use predefined segment labels or choose clustering instead
Preprocessing before split	Leaks distributional information	Fit every transformer on training data/folds
Using customer ID as a feature	Memorizes identity and harms privacy	Retain ID only for traceability
LabelEncoder on input columns	Cannot safely handle unseen values	Use OrdinalEncoder or OneHotEncoder with explicit unknown handling
Standardizing before split	Test information influences transformation	Place scaling inside a pipeline
Negative input to ComplementNB	Violates input requirements	Use a non-negative representation
GaussianNB on arbitrary category codes	Codes imply false numeric distance	Use compatible categorical representation
Different splits for models	Comparison becomes confounded	Use one saved split and identical CV folds
Selecting from test scores	Overfits the final evaluation	Select by training-only CV

Mistake	Why it is incorrect	Correct practice
Reporting accuracy only	Hides minority-class failures	Report macro and class-wise metrics
Treating posterior as certainty	Assumptions may make probabilities overconfident	Calibrate or use cautious review thresholds
Causal business claims	Prediction does not establish causality	Use descriptive, evidence-based language
Autonomous sensitive decisions	Profiling can cause unfair treatment	Require policy review and human oversight
Using a predictor that defines the historical segment label	Creates circular prediction and inflated performance	Audit label provenance; exclude deterministic or post-label proxies.
Treating inferred psychographics as ground truth	Inferred attitudes may be noisy, unstable, or biased	Record measurement provenance and reliability; interpret cautiously.
Declaring one model “better” from a tiny CV difference	Fold variability may be larger than the observed mean gap	Report fold values/SD and confidence intervals in the extension.
Choosing a confidence threshold from the test set	Leaks test information into the decision policy	Select the threshold from validation coverage–error trade-offs and freeze it.
Reporting fairness on tiny subgroups without caveat	Metrics may be statistically unstable and misleading	Report subgroup n and mark insufficient-sample groups explicitly.
Random-split-only evaluation for time-varying behavior	Can hide temporal drift or future-information leakage	Add chronological holdout analysis when timestamps exist.

25. Responsible Analytics

- Customer consent and purpose limitation
- Data minimization and de-identification
- Sensitive-attribute review
- Bias in historical labels
- Fairness across relevant groups
- Risk of stereotyping
- Transparency and explainability
- Human oversight
- Data retention and security
- Prohibition of discriminatory targeting
- Periodic monitoring for drift and degradation

25.2 Quantitative Fairness Extension

When a sensitive or protected attribute is legally and ethically appropriate for auditing, report subgroup sample counts and at least one group-wise metric such as recall, false-positive rate, or macro F1. Do not interpret unstable estimates from very small subgroups; report “insufficient sample” where appropriate. Fairness auditing does not imply that the protected attribute should be used as a predictor.

25.3 Temporal Drift Extension

If timestamps are available, compare a chronological holdout with the random split. Discuss whether customer behaviour, psychographic measurements, or segment definitions drift over time and what monitoring or retraining trigger would be needed.

25.1 Risk-register template

Risk	Cause	Impact	Mitigation	Responsible stakeholder

26. Submission Guidelines

- RegistrationNumber_Lab04_CustomerSegmentation.ipynb
- RegistrationNumber_Lab04_Report.pdf
- RegistrationNumber_Lab04_CV_Results.csv
- RegistrationNumber_Lab04_Test_Results.csv
- RegistrationNumber_Lab04_NewCustomer_Predictions.csv
- RegistrationNumber_Lab04_Error_Analysis.csv
- RegistrationNumber_Lab04_README.md
- models/
- figures/
- artifacts/

The notebook must run from top to bottom in a clean runtime. Do not submit API keys, personal data, or fabricated results.

27. Assessment Rubric – 100 Marks

Component	Marks	Excellent	Satisfactory	Inadequate
Problem framing and technical correctness	6	Correct supervised formulation; prediction unit, target, decision context, and classification-vs-clustering distinction are explicit.	Core framing is correct but decision context or assumptions are only partly justified.	Task is misframed, target is unclear, or Naive Bayes is treated as clustering.
Dataset provenance, label audit, and governance	8	Fixed dataset-pack version/checksum, data dictionary, licence, privacy, label provenance, circularity risks, and psychographic measurement origins are documented.	Most provenance is documented but one of label provenance, checksum, or measurement origin is weak.	Dataset/label origin is unclear, identifiers are retained, or circular predictors are ignored.
EDA and feature taxonomy	7	Class balance, missingness, duplicates, plausible ranges, and demographic/psychographic/behavioral groups are correctly audited and interpreted.	EDA is mostly complete with limited interpretation or one missing audit.	EDA is superficial or feature groups are technically incorrect.
Split design and leakage prevention	10	Saved stratified/grouped split, no ID/group overlap, all preprocessing fit inside training/CV, test remains locked.	Split is valid but manifest/group checks or evidence are incomplete.	Preprocessing precedes split, test influences

Component	Marks	Excellent	Satisfactory	Inadequate
				tuning, or overlap/leakage is present.
Core Naive Bayes implementations	15	Dummy, GaussianNB, BernoulliNB, and fully executable CategoricalNB use technically compatible representations; unknown categories and non-negativity are verified.	All core models run but one representation/assumption is weakly justified.	Required model is missing or receives an incompatible representation.
Feature-group ablation	7	A predeclared core model compares demographic, psychographic, behavioral, and combined groups on identical folds with clear interpretation.	All groups are compared but interpretation or fold consistency is incomplete.	Groups are incomparable, use different splits, or results are missing.
Validation, uncertainty, and model selection	10	Identical folds; macro-F1 mean, SD, fold values, class-wise recall, and assumption/cost considerations support a pre-test selection decision.	CV is correct but variability or decision rationale is incomplete.	Model selected from test results or only accuracy is used.
Locked-test evaluation and visualization	9	One-time test evaluation reports macro/weighted metrics, per-class scores, count and normalized confusion matrices, latency, and clear figures.	Most required metrics/figures are correct with minor omissions.	Repeated test peeking, missing class-wise metrics, or misleading visualizations.
Error, confidence, and review policy	8	At least five errors are tied to business cost; posterior uncertainty is discussed; validation coverage–error trade-off determines a frozen review threshold.	Error analysis is present but threshold selection or operational implications are limited.	No meaningful error analysis or threshold chosen from test data/arbitrarily.
Responsible analytics and fairness	7	Privacy, sensitive attributes, profiling, psychographic reliability, human oversight, and subgroup-audit limitations are addressed; extension reports subgroup n and metrics when appropriate.	Responsible-use discussion is present but lacks quantitative or operational detail.	Sensitive/profiling risks are ignored or discriminatory use is encouraged.
Reproducibility and acceptance tests	7	Notebook runs cleanly; versions, split manifest, fitted model reload, CategoricalNB non-negativity, feature coverage, saved artifacts, and deterministic checks pass.	Most artifacts/tests are present but one reproducibility check is missing.	Notebook depends on hidden state, artifacts are missing, or key tests fail.
Report quality and evidence-based conclusion	6	Concise, well-structured report separates validation from test evidence, avoids causal overclaiming, discusses limitations/drift, and recommends suitable/unsuitable use.	Report is understandable but some claims are weakly supported or limitations are brief.	Report is unclear, unsupported, or overstates deployment readiness.

Performance bands: 90–100 Exceptional; 80–89 Very Good; 70–79 Good; 60–69 Satisfactory; below 60 Insufficient.

28. Final Submission Checklist

- ☐ Target exists and contains at least two classes.
- ☐ Customer IDs are not predictors.
- ☐ Direct identifiers are removed.
- ☐ Train/test IDs do not overlap.
- ☐ Preprocessing is fitted only on training data.
- ☐ All candidate models use the same split and folds.
- ☐ Each model receives a compatible representation.
- ☐ Dummy baseline is reported.
- ☐ Model selection uses cross-validation rather than test results.
- ☐ Macro F1 and class-wise metrics are included.
- ☐ Locked test is evaluated once after selection.
- ☐ At least five errors are interpreted.
- ☐ New-customer prediction validates input.
- ☐ Confidence is not described as certainty.
- ☐ Privacy and fairness risks are documented.
- ☐ All required artifacts exist.
- ☐ Notebook runs from top to bottom.
- ☐ Rubric evidence is easy to locate.
- ☐ Fixed dataset-pack version/checksum, data dictionary, and label-definition note are recorded.
- ☐ Label-provenance/circularity audit is completed; deterministic or post-label proxies are excluded.
- ☐ Psychographic variables have measurement provenance (self-reported/survey-derived/inferred) documented.
- ☐ Core classifiers are Dummy, GaussianNB, BernoulliNB, and CategoricalNB; CategoricalNB unknown values map to non-negative codes.
- ☐ ComplementNB is used only as a justified extension, not as an assumed default for tabular customer data.
- ☐ Fold-level macro F1, mean, and SD are saved; small score differences are interpreted cautiously.
- ☐ Review/abstention threshold is selected from validation coverage–error trade-offs and frozen before test evaluation.
- ☐ Extension fairness results report subgroup sample sizes and avoid claims for insufficient-sample groups.
- ☐ Temporal holdout/drift analysis is completed when timestamps are available (extension).
- ☐ Saved model reload reproduces predictions for a small validation fixture.
- ☐ At least three public customer datasets are documented with verified source URLs and suitability notes.
- ☐ The assessed core uses a dataset with a valid predefined supervised target; no labels are fabricated.
- ☐ Advanced learners compare the selected Naive Bayes model with TabTransformer/FT-Transformer on the same frozen split.
- ☐ BERT/DistilBERT is used only if genuine customer text is available, with text-only and tabular-only ablations where applicable.

29. References and Student-Help Links

Resource	Link
scikit-learn: Naive Bayes	https://scikit-learn.org/stable/modules/naive_bayes.html

Resource	Link
GaussianNB	https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html
CategoricalNB	https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.CategoricalNB.html
BernoulliNB	https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.BernoulliNB.html
ComplementNB	https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.ComplementNB.html
ColumnTransformer	https://scikit-learn.org/stable/modules/compose.html#columntransformer-for-heterogeneous-data
Pipeline	https://scikit-learn.org/stable/modules/compose.html#pipeline
Model selection	https://scikit-learn.org/stable/model_selection.html
Classification metrics	https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics
Google ML Crash Course: Classification	https://developers.google.com/machine-learning/crash-course/classification
NIST AI Risk Management Framework	https://www.nist.gov/itl/ai-risk-management-framework
Stanford CS229 – Project guidance: baseline, preliminary experiments, error analysis	https://cs229.stanford.edu/projects.html-backup-fall21
MIT 6.390 / SERC – weekly lab check-offs and responsible-computing prompts	https://ocw.mit.edu/courses/res-tll-008-social-and-ethical-responsibilities-of-computing-serc/pages/privacy-surveillance/introduction-to-machine-learning/
UC Berkeley CS 289A – sound experiments, train/validation/test, CV, error bars	https://people.eecs.berkeley.edu/~jrs/189s22/project.html
CMU Machine Learning in Practice – representation, interpretation, error analysis	https://www.lti.cs.cmu.edu/misc-pages/intranet-course-info.html

Record access dates, package versions, dataset licences, and all external sources used. Adapt examples only after understanding and testing them.

Verified dataset resources (checked August 2026)

- Analytics Vidhya JanataHack Customer Segmentation:
<https://www.analyticsvidhya.com/datahack/contest/janatahack-customer-segmentation>
- Kaggle mirror - Customer Segmentation (A-D target): <https://www.kaggle.com/datasets/vettrirah/customer>
- Kaggle Customer Personality Analysis: <https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis>
- UCI Online Retail II, DOI 10.24432/C5CG6D: <https://archive.ics.uci.edu/dataset/502/online+retail+ii>
- UCI Bank Marketing: <https://archive.ics.uci.edu/dataset/222/bank+marketing>

Advanced Transformer resources

- Keras TabTransformer structured-data example: https://keras.io/examples/structured_data/tabtransformer/
- Huang et al. TabTransformer: Tabular Data Modeling Using Contextual Embeddings, arXiv:2012.06678.
- Gorishniy et al. Revisiting Deep Learning Models for Tabular Data (FT-Transformer), NeurIPS 2021.
- Hugging Face Transformers - Text Classification:
https://huggingface.co/docs/transformers/tasks/sequence_classification

Appendix A. Complete Executable Notebook Scaffold

A1. Environment, versions, and output folders

```
# 0. Optional installation (Colab)
# !pip install -q pandas numpy scikit-learn matplotlib joblib

from pathlib import Path
import json, platform, sys, time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
from joblib import dump

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, KBinsDiscretizer, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.dummy import DummyClassifier
from sklearn.naive_bayes import GaussianNB, CategoricalNB, BernoulliNB, ComplementNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, accuracy_score, precision_recall_fscore_support,
                             f1_score, log_loss)

SEED = 42
np.random.seed(SEED)
OUT = Path('lab04_outputs')
for d in ['figures', 'models', 'artifacts', 'results']:
    (OUT/d).mkdir(parents=True, exist_ok=True)

versions = {
    'python': sys.version,
    'platform': platform.platform(),
    'pandas': pd.__version__,
    'numpy': np.__version__,
    'scikit_learn': sklearn.__version__,
}
(OUT/'artifacts'/'versions.json').write_text(json.dumps(versions, indent=2))
print(versions)
```

A2. Load and validate the dataset

```
# 1. Load the instructor-provided dataset
DATA_PATH = Path('/content/customer_segmentation.csv') # MODIFY
TARGET = 'customer_segment' # MODIFY if needed
ID_COL = 'customer_id' # MODIFY if needed

if not DATA_PATH.exists():
```

```

    raise FileNotFoundError(f'Dataset not found: {DATA_PATH}')

df = pd.read_csv(DATA_PATH)
required = {TARGET, ID_COL}
missing = required - set(df.columns)
if missing:
    raise ValueError(f'Missing required columns: {sorted(missing)}')
if df[TARGET].nunique() < 2:
    raise ValueError('Target must contain at least two classes.')
if df[ID_COL].duplicated().any():
    print('Warning: duplicate customer IDs detected; investigate before splitting.')

print(df.shape)
display(df.head())
print(df[TARGET].value_counts(dropna=False))

```

A3. Declare feature groups

```

# 2. Configure the actual feature groups from the dataset
DEMOGRAPHIC = ['age', 'education', 'occupation', 'income', 'family_size', 'geographic_region']
PSYCHOGRAPHIC = ['lifestyle', 'price_sensitivity', 'brand_consciousness', 'technology_affinity']
BEHAVIORAL = ['purchase_frequency', 'average_order_value', 'total_spending',
              'recency', 'discount_usage', 'campaign_response', 'engagement_score']

# Keep only columns that really exist; document any omissions.
def existing(cols):
    return [c for c in cols if c in df.columns]
DEMOGRAPHIC, PSYCHOGRAPHIC, BEHAVIORAL = map(existing,
        [DEMOGRAPHIC, PSYCHOGRAPHIC, BEHAVIORAL])
ALL_FEATURES = list(dict.fromkeys(DEMOGRAPHIC + PSYCHOGRAPHIC + BEHAVIORAL))
if not ALL_FEATURES:
    raise ValueError('No configured predictors exist in the dataset.')
if ID_COL in ALL_FEATURES:
    raise AssertionError('Customer ID must not be a predictor.')

feature_manifest = {
    'demographic': DEMOGRAPHIC,
    'psychographic': PSYCHOGRAPHIC,
    'behavioral': BEHAVIORAL,
    'all_features': ALL_FEATURES,
}
(OUT/'artifacts'/'feature_manifest.json').write_text(json.dumps(feature_manifest, indent=2))
print(json.dumps(feature_manifest, indent=2))

```

A4. Data-quality and class-distribution audit

```

# 3. Basic audit
summary = pd.DataFrame({
    'dtype': df[ALL_FEATURES + [TARGET]].dtypes.astype(str),
    'missing_count': df[ALL_FEATURES + [TARGET]].isna().sum(),
    'missing_percent': 100 * df[ALL_FEATURES + [TARGET]].isna().mean(),
    'unique_count': df[ALL_FEATURES + [TARGET]].nunique(dropna=False),
})
summary.to_csv(OUT/'results'/'data_audit.csv')
display(summary)

print('Exact duplicate rows:', df.duplicated().sum())
print('Duplicate feature profiles:', df.duplicated(subset=ALL_FEATURES).sum())

class_counts = df[TARGET].value_counts().sort_index()
ax = class_counts.plot(kind='bar', title='Customer Segment Distribution')
ax.set_xlabel('Segment'); ax.set_ylabel('Number of customers')
plt.tight_layout(); plt.savefig(OUT/'figures'/'class_distribution.png', dpi=180); plt.show()

```

A5. Create and save the locked split

```

# 4. Locked stratified split
usable = df.dropna(subset=[TARGET]).copy()
train_df, test_df = train_test_split(
    usable, test_size=0.20, random_state=SEED, stratify=usable[TARGET]
)
assert set(train_df[ID_COL]).isdisjoint(set(test_df[ID_COL]))

split_manifest = pd.concat([
    train_df[[ID_COL]].assign(split='train'),
    test_df[[ID_COL]].assign(split='test')
], ignore_index=True)
split_manifest.to_csv(OUT/'artifacts'/'split_manifest.csv', index=False)

X_train = train_df[ALL_FEATURES]
y_train = train_df[TARGET]
X_test = test_df[ALL_FEATURES]

```

```
y_test = test_df[TARGET]
print(X_train.shape, X_test.shape)
```

A6. Verify numerical, categorical, and binary feature lists

```
# 5. Infer data-type groups, then verify manually
numeric_cols = [c for c in ALL_FEATURES if pd.api.types.is_numeric_dtype(df[c])]
categorical_cols = [c for c in ALL_FEATURES if c not in numeric_cols]

# Set binary columns explicitly after inspecting values.
binary_cols = [c for c in categorical_cols
                if set(df[c].dropna().astype(str).unique()).issubset({'0', '1', 'False', 'True', 'No', 'Yes'})]
nominal_cols = [c for c in categorical_cols if c not in binary_cols]
print('Numeric:', numeric_cols)
print('Nominal:', nominal_cols)
print('Binary:', binary_cols)
```

A7. Build leakage-safe common pipelines

6. Core leakage-safe pipelines

```
from sklearn.base import BaseEstimator, TransformerMixin
```

```
class SafeOrdinalToNonNegative(BaseEstimator, TransformerMixin):
```

```
    """Encode known categories as 1..K and reserve 0 for unseen categories."""
```

```
    def __init__(self):
```

```
        self.enc = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
```

```
    def fit(self, X, y=None):
```

```
        self.enc.fit(X)
```

```
        return self
```

```
    def transform(self, X):
```

```
        return self.enc.transform(X).astype(int) + 1
```

BernoulliNB: discretized numeric variables + one-hot categorical variables

```
numeric_binary = Pipeline([
```

```
    ('imputer', SimpleImputer(strategy='median')),
```

```
    ('bins', KBinsDiscretizer(n_bins=5, encode='onehot', strategy='quantile')),
```

```
])
```

```
category_ohe = Pipeline([
```

```
    ('imputer', SimpleImputer(strategy='most_frequent')),
```

```
    ('onehot', OneHotEncoder(handle_unknown='ignore')),
```

```
])
```

```
bernoulli_preprocessor = ColumnTransformer([
```

```
    ('num_bins', numeric_binary, numeric_cols),
```

```
    ('cat', category_ohe, nominal_cols + binary_cols),
```

```
], remainder='drop')
```

CategoricalNB: discretized numeric variables + safe non-negative integer category codes

```
cat_num = Pipeline([
```

```
    ('imputer', SimpleImputer(strategy='median')),
```

```
    ('bins', KBinsDiscretizer(n_bins=5, encode='ordinal', strategy='quantile')),
```

```
])
```



```

cat_cat = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('safe_ordinal', SafeOrdinalToNonNegative()),
])
categorical_nb_preprocessor = ColumnTransformer([
    ('num', cat_num, numeric_cols),
    ('cat', cat_cat, nominal_cols + binary_cols),
], remainder='drop')

core_pipelines = {
    'Dummy_most_frequent': Pipeline([
        ('prep', bernoulli_preprocessor),
        ('model', DummyClassifier(strategy='most_frequent')),
    ]),
    'BernoulliNB': Pipeline([
        ('prep', bernoulli_preprocessor),
        ('model', BernoulliNB(alpha=1.0, binarize=0.0)),
    ]),
    'CategoricalNB_mixed': Pipeline([
        ('prep', categorical_nb_preprocessor),
        ('model', CategoricalNB(alpha=1.0)),
    ]),
}

# GaussianNB uses continuous numerical variables only.
if numeric_cols:
    gaussian_preprocessor = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
    ])
    core_pipelines['GaussianNB_numeric_only'] = Pipeline([
        ('prep', ColumnTransformer([
            ('num', gaussian_preprocessor, numeric_cols)
        ], remainder='drop')),
        ('model', GaussianNB(var_smoothing=1e-9)),
    ])

# Verify CategoricalNB support constraints using TRAINING data only.
Xt_cat = categorical_nb_preprocessor.fit_transform(X_train)
assert np.nanmin(np.asarray(Xt_cat)) >= 0, 'CategoricalNB received a negative category code.'

```

Research-extension models are added only after the core workflow is complete.

```
extension_models = {
    'ComplementNB_extension': ComplementNB(alpha=1.0),
    'LogisticRegression_extension': LogisticRegression(max_iter=2000, class_weight='balanced'),
}
```

A10. Training-only cross-validation

```
# 9. Cross-validation comparison using identical folds
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
scoring = {'accuracy': 'accuracy', 'macro_f1': 'f1_macro', 'weighted_f1': 'f1_weighted'}
rows = []
for name, pipe in core_pipelines.items():
    start = time.perf_counter()
    scores = cross_validate(pipe, X_train, y_train, cv=cv, scoring=scoring,
                           return_train_score=False, n_jobs=-1)
    elapsed = time.perf_counter() - start
    rows.append({
        'model': name,
        'accuracy_mean': scores['test_accuracy'].mean(),
        'macro_f1_mean': scores['test_macro_f1'].mean(),
        'macro_f1_sd': scores['test_macro_f1'].std(ddof=1),
        'weighted_f1_mean': scores['test_weighted_f1'].mean(),
        'cv_time_seconds': elapsed,
    })
cv_results = pd.DataFrame(rows).sort_values('macro_f1_mean', ascending=False)
cv_results.to_csv(OUT/'results'/ 'cv_results.csv', index=False)
display(cv_results)
```

A11. Locked-test evaluation

```
# 10. Select before inspecting the test set
selected_name = cv_results.iloc[0]['model']
selected_model = core_pipelines[selected_name]
print('Selected by mean CV macro F1:', selected_name)

start = time.perf_counter()
selected_model.fit(X_train, y_train)
train_seconds = time.perf_counter() - start

start = time.perf_counter()
y_pred = selected_model.predict(X_test)
inference_seconds = time.perf_counter() - start

report = classification_report(y_test, y_pred, output_dict=True, zero_division=0)
pd.DataFrame(report).T.to_csv(OUT/'results'/ 'classification_report.csv')
print(classification_report(y_test, y_pred, zero_division=0))
print('Accuracy:', accuracy_score(y_test, y_pred))
print('Macro F1:', f1_score(y_test, y_pred, average='macro'))
print('Weighted F1:', f1_score(y_test, y_pred, average='weighted'))
print('Training seconds:', train_seconds)
print('Inference seconds:', inference_seconds)

ConfusionMatrixDisplay.from_predictions(y_test, y_pred, xticks_rotation=45, cmap='Blues')
plt.title(f'Confusion Matrix - {selected_name}')
plt.tight_layout(); plt.savefig(OUT/'figures'/ 'confusion_matrix.png', dpi=180); plt.show()
```

A12. Save predictions and model artifacts

```
# 11. Save predictions, errors, and the fitted pipeline
pred_df = test_df[['ID_COL', 'TARGET']].copy()
pred_df['predicted_segment'] = y_pred

if hasattr(selected_model, 'predict_proba'):
    probs = selected_model.predict_proba(X_test)
    classes = selected_model.classes_
    pred_df['max_posterior'] = probs.max(axis=1)
    pred_df['confidence_category'] = pd.cut(
        pred_df['max_posterior'], bins=[-np.inf, .50, .75, np.inf],
        labels=['low_review', 'moderate_review', 'high']
    )
    for i, cls in enumerate(classes):
        pred_df[f'prob_{cls}'] = probs[:, i]
else:
    pred_df['max_posterior'] = np.nan
    pred_df['confidence_category'] = 'decision_score_only'
```

```
pred_df.to_csv(OUT/'results'/'test_predictions.csv', index=False)
pred_df[pred_df[TARGET] != pred_df['predicted_segment']].to_csv(
    OUT/'results'/'error_analysis.csv', index=False)
dump(selected_model, OUT/'models'/'selected_pipeline.joblib')
print(pred_df.head())
```

A13. New-customer prediction function

```
# 12. New-customer prediction helper
REQUIRED_INPUTS = ALL_FEATURES

def predict_customer_segment(customer_profile: dict) -> dict:
    missing = [c for c in REQUIRED_INPUTS if c not in customer_profile]
    if missing:
        raise ValueError(f'Missing mandatory fields: {missing}')
    one = pd.DataFrame([customer_profile], columns=REQUIRED_INPUTS)
    pred = selected_model.predict(one)[0]
    result = {'predicted_segment': str(pred)}
    if hasattr(selected_model, 'predict_proba'):
        p = selected_model.predict_proba(one)[0]
        distribution = {str(c): float(v) for c, v in zip(selected_model.classes_, p)}
        confidence = float(p.max())
        review = 'normal_review' if confidence >= .75 else (
            'explicit_review' if confidence >= .50 else 'manual_analysis')
        result.update({
            'posterior_distribution': distribution,
            'max_posterior': confidence,
            'review_recommendation': review,
        })
    else:
        result['review_recommendation'] = 'manual_review_no_probability'
    return result

# Replace with a valid synthetic or de-identified profile.
# example = {feature: ... for feature in REQUIRED_INPUTS}
# print(predict_customer_segment(example))
```

A14. Core acceptance tests

```
# 13. Acceptance tests and artifact verification
assert TARGET in df.columns
assert df[TARGET].nunique() >= 2
assert ID_COL not in ALL_FEATURES
assert set(train_df[ID_COL]).isdisjoint(set(test_df[ID_COL]))
assert set(np.unique(y_pred)).issubset(set(y_train.unique()))
assert (OUT/'models'/'selected_pipeline.joblib').exists()
assert (OUT/'results'/'cv_results.csv').exists()
assert (OUT/'results'/'test_predictions.csv').exists()
assert set(DEMOGRAPHIC + PSYCHOGRAPHIC + BEHAVIORAL).issubset(set(ALL_FEATURES))

assert ID_COL not in set(ALL_FEATURES)

assert len(cv_results) == len(core_pipelines)

required_core = {'Dummy_most_frequent', 'BernoulliNB', 'CategoricalNB_mixed'} | ({'GaussianNB_numeric_only'} if
numeric_cols else set())
assert required_core.issubset(set(cv_results['model']))

reloaded = __import__('joblib').load(OUT/'models'/'selected_pipeline.joblib')

assert np.array_equal(reloaded.predict(X_test.head(min(5, len(X_test)))), selected_model.predict(X_test.head(min(5,
len(X_test)))))

print('Core acceptance tests: PASSED')
```

Appendix B. Research Extensions: ComplementNB, Uncertainty, Fairness, and Temporal Shift

Complete the following only after the core notebook is reproducible: (1) ComplementNB on a justified sparse non-negative representation; (2) Logistic Regression benchmark; (3) repeated stratified CV or bootstrap confidence interval; (4) validation coverage–error threshold analysis; (5) quantitative subgroup audit with sample-size reporting; and (6) chronological holdout when timestamps exist.

```
from sklearn.base import BaseEstimator, TransformerMixin

class SafeOrdinalToNonNegative(BaseEstimator, TransformerMixin):
```

```

def __init__(self):
    self.enc = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
def fit(self, X, y=None):
    self.enc.fit(X)
    return self
def transform(self, X):
    # Known values start at 0; unseen value -1 becomes 0 after +1.
    return self.enc.transform(X).astype(int) + 1

categorical_nb_preprocessor = ColumnTransformer([
    ('num', Pipeline([
        ('impute', SimpleImputer(strategy='median')),
        ('bin', KBinsDiscretizer(n_bins=5, encode='ordinal', strategy='quantile')),
    ]), numeric_cols),
    ('cat', Pipeline([
        ('impute', SimpleImputer(strategy='most_frequent')),
        ('safe_ordinal', SafeOrdinalToNonNegative()),
    ]), nominal_cols + binary_cols),
], remainder='drop')

categorical_nb_pipe = Pipeline([
    ('prep', categorical_nb_preprocessor),
    ('model', CategoricalNB(alpha=1.0)),
])

# Validate transformed values before training.
Xt = categorical_nb_preprocessor.fit_transform(X_train)
assert np.all(np.asarray(Xt) >= 0)

```

Appendix C. Feature-Group Experiment Function

```

def evaluate_feature_group(group_name, columns, model_factory, cv):
    if not columns:
        return {'feature_group': group_name, 'status': 'no configured columns'}
    Xg = train_df[columns]
    pipe = model_factory(columns)
    scores = cross_validate(pipe, Xg, y_train, cv=cv,
                           scoring={'macro_f1': 'f1_macro', 'weighted_f1': 'f1_weighted'},
                           n_jobs=-1)

    return {
        'feature_group': group_name,
        'macro_f1_mean': scores['test_macro_f1'].mean(),
        'macro_f1_sd': scores['test_macro_f1'].std(ddof=1),
        'weighted_f1_mean': scores['test_weighted_f1'].mean(),
    }

# Build model_factory so it creates a fresh compatible pipeline for the supplied columns.
# Run for demographic, psychographic, behavioral, and combined groups using identical CV folds.

```

Appendix D. Report and Interpretation Guidance

- State one clear research question and one testable hypothesis.
- Describe the dataset and feature groups without revealing personal data.
- Explain why each model is compatible with its representation.
- Report validation results before test results.
- Interpret at least five errors and the implications of minority-class recall.
- Discuss whether performance justifies the inclusion of sensitive features.
- Separate correlation and prediction from causal claims.
- Conclude with suitable use, unsuitable use, limitations, and monitoring needs.

Appendix E. Quality-Assurance Summary

Check	Status
Task correctly framed as supervised classification	Included
Multiple compatible Naive Bayes variants	Included
Baseline-first workflow	Included
Leakage-safe split and pipelines	Included

Check	Status
Feature-group ablation	Included
Macro and class-wise evaluation	Included
Confidence and review policy	Included
Privacy, fairness, and misuse controls	Included
Executable notebook scaffold and acceptance tests	Included
Three-hour schedule and 100-mark rubric	Included
Three-hour core reduced to four core models plus one feature-group ablation	Included in Revision 2.0
Complete executable CategoricalNB unknown-category handling	Included in core appendix/code path
Fixed dataset pack, label provenance, and circularity audit	Included
Fold variability + extension bootstrap uncertainty	Included
Validation-selected coverage–error review threshold	Included
Quantitative fairness and temporal-shift extensions	Included
Criterion-specific 100-mark rubric	Included

Revision 2.0 quality target and post-revision rating: 9.4/10. The three-hour core is feasible because it is limited to four core classifiers and one feature-group ablation; advanced uncertainty, ComplementNB, Logistic Regression, quantitative fairness, calibration, and temporal robustness are extensions. Classroom readiness still depends on a fixed instructor-issued dataset pack with verified schema, checksum, label provenance, and data dictionary.

Appendix E2. Extension Result Templates

E2.1 Uncertainty and robustness

Model	Validation scheme	Macro F1 mean	SD	95% CI	Interpretation

E2.2 Confidence / selective prediction

Threshold	Coverage	Selective error	Review rate	Business interpretation

E2.3 Quantitative fairness audit

Audit group	n	Recall	False-positive rate	Macro F1	Stability caveat

E2.4 Temporal robustness

Train period	Test period	Macro F1	Change vs random split	Drift interpretation

Appendix F. Revision 2.0 Benchmark Alignment Summary

The revised design reflects recurring practices visible in Stanford CS229 project milestones (baseline and preliminary error analysis), MIT 6.390/SERC labs (instructor check-offs and responsible-computing reflection), UC Berkeley CS 289A project guidance (train/validation/test separation, comparative experiments, cross-validation, error bars), and CMU practical ML guidance (representation choice, interpretation, and error analysis). These references are benchmarks for pedagogy, not claims of institutional endorsement.

Appendix G. Advanced Transformer Extension

G.1 Recommended architecture choice

Structured-only customer data: use TabTransformer or FT-Transformer. Mixed tabular plus genuine free text: use a tabular Transformer together with a BERT/DistilBERT text encoder and fuse representations before classification. Text-only data: use BERT/DistilBERT sequence classification.

G.2 Minimum advanced experiment

Use Dataset A and exactly the same split manifest as the Naive Bayes study. Tune on training/validation data only. Report locked-test macro F1, weighted F1, per-class recall, confusion matrix, training time, inference latency, parameter count, and at least five error cases.

G.3 Fair-comparison rules

Keep identical target labels, split indices, and evaluation metrics. Record CPU/GPU environment and random seeds. Do not claim improvement without reporting variability or repeated runs. A Transformer that is slower and only marginally better should be discussed as a trade-off rather than automatically selected.

G.4 Optional BERT fusion design

If genuine customer text exists, tokenize it with a pretrained BERT-family tokenizer, obtain a pooled/CLS representation or fine-tuned encoder output, concatenate that representation with processed tabular features, and train a small classification head. Compare text-only, tabular-only, and fused variants to quantify incremental value from text.